Typy danych w PHP i operatory

Język PHP udostępnia osiem typów danych: jeden typ logiczny, dwa typy liczbowe, dwa typy specjalne, dwa typy złożone oraz typ łańcuchowy. PHP nie wymaga jawnego zadeklarowania typu danych i pozwala na dynamiczną jego zmianę w trakcie wykonywania skryptu.
Typ logiczny może przyjmować jedną z dwóch wartości logicznych: true (prawda) oraz false (fałsz). Najczęściej stosowany jest podczas sprawdzania warunków logicznych w instrukcjach warunkowych i pętlach.
Typ liczbowy wykorzystywany jest do deklaracji liczb całkowitych i zmiennoprzecinkowych. Liczby całkowite mogą przyjmować wartości dodatnie, ujemne oraz zero. Mogą zostać zapisane w notacji dziesiętnej, ósemkowej lub szesnastkowej. Należy pamiętać, że liczby całkowite i zmiennoprzecinkowe mają ograniczenia co do wielkości w zależności od platformy, na której działa PHP.
Typy specjalne to: wartość NULL oznaczająca, że do zmiennej nie przypisano żadnej wartości, oraz identyfikator zasobów, który ma odwołanie do zewnętrznego źródła danych (plik, baza danych).
Typ złożony określa dwa z najbardziej rozbudowanych typów danych. Pierwszy z nich to tablica, która służy do przechowywania większej ilości danych. Drugi typ to obiekty mające odpowiednie właściwości i własne metody (funkcje).
Typ łańcuchowy odpowiada za przechowywanie ciągów znaków (łańcuchów). Ciąg należy umieścić pomiędzy znakami apostrofów lub cudzysłowów.

Typy wybranych zmiennych w PHP

- liczby całkowite (integer)
- liczby rzeczywiste (double)
- ciągi (string)
- tablice (array)
- obiekty (object)

Dodatkowo PHP potrafi konwertować zmienne całkowite zapisane w różnych formatach liczbowych.

```
<?php
$a = 1234; # liczba dziesiętna
$a = -123; # liczba ujemna
$a = 0123; # liczba ósemkowa (równoznaczne z
dziesiętnym 83)
$a = 0x12; # liczba szesnastkowa (równoznaczne
z dziesiętnym 18)
?>
```

Na zmiennych i stałych mogą być wykonywane operacje z wykorzystaniem różnych operatorów. Język PHP udostępnia następujące operatory: arytmetyczne, logiczne, bitowe, przypisania, porównania, inkrementacji, dekrementacji oraz operator obsługi błędów i operator ciągu.

Operatory arytmetyczne

Przykład	Nazwa	Wynik
$a + $b	Dodawanie	Suma $a i $b.
$a – $b	Odejmowanie	Różnica $a i $b.
$a * $b	Mnożenie	Iloczyn $a i $b.
$a / $b	Dzielenie	Iloraz $a i $b (bez reszty).
$a % $b	Modulo	Reszta z dzielenia $a przez $b.

Operatory przypisania

Przykład	Wynik
$a += 2	Do zmiennej $a dodane zostanie 2
$a -= 2	Od zmiennej $a odjęte zostanie 2
$a *= 2	Zmienna $a zostanie pomnożona przez 2
$a /= 2	Zmienna $a podzielona przez 2
$a %= 2	Zmienna $a przyjmie wartość reszty z dzielenia $a przez 2
$a .= ” dalszy ciąg”	Do zmiennej $a na końcu dodany zostanie ciąg ” dalszy ciąg”

```
<?php
$nazwa = $inna_nazwa = $trzecia_nazwa = 5;
?>
```

W tym wypadku wszystkim zmiennym zostanie przypisana wartość 5. Operator przypisania można łączyć z operatorami arytmetycznymi i operatorem łączenia ciągów

Operatory operacji bitowych

Operatory operacji bitowych pozwalają na przestawianie pojedynczych bitów zmiennych.

Przykład	Nazwa	Wynik
$a & $b	AND	Ustawiane są bity które są ustawione w obu zmiennych.
$a | $b	OR	Ustawiane są bity, które są ustawione w jednej lub drugiej zmiennej.
$a ^ $b	XOR	Ustawiane są bity, które są ustawione w jednej lub drugiej zmiennej, ale nie w obu.
~ $a	NOT	Inwerter – ustawiane są bity które nie są ustawione w zmiennej $a i odwrotnie.
$a << $b	Przesunięcie w lewo	Przesuń bity z $a $b-razy w lewo (każdy krok oznacza pomnożenie przez 2)
$a >> $b	Przesunięcie w prawo	Przesuń bity z $a $b-razy w prawo (każdy krok oznacza podzielenie przez 2)

Operatory porównania

Operatory porównania są niezbędne do korzystania z instrukcji warunkowych (jeśli coś to zrób coś). Zwracają one wartość TRUE (prawda – 1) lub FALSE (fałsz – 0).

Przykład	Nazwa	Wynik
$a == $b	Równy	Prawda jeśli $a jest równe $b.
$a ===	Identyczny	Prawda jeśli $a jest równe $b i są

$b		tego samego typu.
$a != $b	Nie równe	Prawda jeśli $a nie jest równe $b.
$a !== $b	Nie identyczny	Prawda jeśli $a nie jest równe $b lub nie są tego samego typu.
$a < $b	Mniejsze	Prawda jeśli $a jest mniejsze niż $b.
$a > $b	Większe	Prawda jeśli $a jest większe niż $b.
$a <= $b	Mniejsze lub równe	Prawda jeśli $a jest mniejsze lub równe $b.
$a >= $b	Większe lub równe	Prawda jeśli $a jest większe lub równe $b.

Operatory inkrementacji i dekrementacji

Operatory te występują w większości języków programowania. Służą one do zmniejszenia lub zwiększenia wartości danej zmiennej o 1.

Każdy operator można stosować na 2 sposoby:

preinkrementacja/predekrementacja – najpierw wartość zmiennej zostanie zmieniona, a później zwrócona, lub

postinkrementacji/postdekrementacji – najpierw zostanie zwrócona wartość zmiennej, a następnie wartość zmiennej zostanie zmieniona.

Przykład	Nazwa	Wynik
++$a	Preinkrementacja	Zwiększa $a o jeden, a następnie zwraca $a.
$a++	Postinkrementacja	Zwraca $a, a następnie zwiększa $a o jeden.
--$a	Predekrementacja	Zmniejsza $a o jeden, po czym zwraca $a.
$a--	Postdekrementacja	Zwraca $a, po czym zmniejsza $a o jeden.

Operatory logiczne

Operatory logiczne służą do budowania bardziej skomplikowanych instrukcji warunkowych – do łączenia kilku warunków w jednej instrukcji.

Operatory logiczne: koniunkcja (&&), alternatywa (| |) oraz negacja (!) zwracają wartość true (prawda) lub false (fałsz).

Przykład	Nazwa	Wynik
$a && $b	AND	Prawda, jeśli $a i $b są prawdą
$a || $b	OR	Prawda, jeśli $a lub $b są prawdą
! $a	NOT	Prawda, jeśli $a nie jest prawdą

Operator ciągu

Operator ciągu („.' – kropka) służy do łączenia kilku ciągów w jedną całość.

Przykład użycia operatora ciągu

```
<?php

   $zmienna1 = "Wartość zmiennej 'zmienna2'
to";

   $zmienna2 = 5;

   echo $zmienna1." ".$zmienna2; // Powinno się
wyświetlić:
                                 // "Wartość
zmiennej 'zmienna2' to 5"

?>
```

W języku PHP występuje dodatkowo operator kontroli błędów oznaczany znakiem tzw. małpy @. Zastosowanie go przed wybranym wyrażeniem spowoduje, że w wypadku wygenerowania błędu przez to wyrażenie na stronie nie pojawi się ostrzeżenie ani żadna informacja o błędzie. Operator ten można umieszczać przed zmiennymi, stałymi, instrukcjami include () oraz wywołaniami funkcji. Nie wolno stosować go przed definicją funkcji i klasy oraz przed strukturami kontrolnymi, takimi jak **if, While** czy foreach.